

Introduction to Deep Learning – Final Project 2026
Bring Van Gogh Back to Life: Style Transfer & Classification

Nadav Amit, Guy Galanti, Yair Shapira



Part 1: Classification - Discussion & Analysis

Dataset Overview

The dataset utilized for this project focuses on the Post-Impressionism art movement, comprising a total of 6,444 images. The primary objective was to perform a binary classification task: distinguishing paintings by Vincent van Gogh (Class 1) from those by other artists of the same movement (Class 0). The data exhibits a significant class imbalance, with Van Gogh's works (1,004 images) constituting approximately 15.6% of the total, while works by other artists (5,440 images) make up the vast majority. The ratio of non-Van Gogh to Van Gogh images is roughly 5.4:1.

To address this disparity and prevent the model from biasing toward the majority class, we implemented a Weighted Cross Entropy Loss during training with class weights set to [1.0, 5.0]. This strategy ensured that the penalty for misclassifying a Van Gogh painting was significantly higher than for misclassifying a non-Van Gogh one, forcing the model to pay closer attention to the minority class.

Models Introduction

We employed Transfer Learning using two distinct architectures pre-trained on ImageNet: VGG-19 and AlexNet. The two models offer contrasting approaches to feature extraction.

AlexNet, a pioneering architecture from 2012, is relatively shallow (8 layers) and utilizes large receptive fields (e.g., 11 times 11 kernels in the first layer), making it effective for capturing broad features but potentially less sensitive to fine textural details. In contrast, VGG-19 from 2014, is a much deeper network (19 layers) that relies exclusively on small 3 times 3 convolution filters. This depth allows VGG-19 to learn a more hierarchical and complex representation of image textures and styles, which is theoretically advantageous for fine-grained art classification.

Transformations & Augmentation

To prepare the data and ensure model robustness, we implemented a pipeline that leverages Data Augmentation. This technique is fundamental in deep learning, particularly when dealing with limited or imbalanced datasets. By artificially expanding the training set through random perturbations, augmentation forces the model to learn invariant, generalized features, such as brushstroke textures and color distributions, rather than memorizing specific pixel arrangements (overfitting).

Specifically, our pipeline included the following steps: Images were resized to 256 pixels and then randomly cropped to 224 pixels to standardize input sizes while introducing spatial variance. We also incorporated `RandomHorizontalFlip()`, leveraging the fact that artistic style, such as brushstrokes and color palettes, remains invariant under reflection, thereby effectively doubling the training set and mitigating overfitting. Finally, all inputs were normalized using the standard ImageNet mean and standard deviation (mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225]) to align the input distribution with what the pre-trained weights expect.

Training & Tuning

Our training strategy adopted a Feature Extraction approach, leveraging the power of Transfer Learning. We utilized the pre-trained weights of VGG-19 and AlexNet, which were originally optimized on the massive ImageNet dataset (1.2 million images). The core hypothesis driving this choice is that the visual features learned by these networks on natural images, such as edge detection, texture recognition, and shape composition, are highly transferable to the domain of art. Specifically, early layers in CNNs learn universal visual primitives (e.g., Gabor filters, color blobs) that are as relevant to a Van Gogh painting as they are to a photograph of a cat. Implementation involved a strict freezing protocol:

Frozen Backbone: We completely froze the gradients for all convolutional blocks (the "backbone"). This ensures that the rich, hierarchical feature maps learned from ImageNet are preserved and not destroyed or "catastrophically forgotten" during the initial training phases on our smaller dataset.

Trainable Classifier Head: We replaced the original 1000-class output layer with a new, randomly initialized Fully Connected (FC) layer tailored to our binary task (Van Gogh vs. Non-Van Gogh). Only the weights of this final classification head were updated during backpropagation.

To determine the optimal configuration for our classifiers, we conducted a time-constrained hyperparameter search using Optuna, integrated with Weights & Biases (W&B) for real-time visualization. Given resource constraints, we allocated a strictly balanced time budget of 30 minutes for each architecture (VGG-19 and AlexNet). This approach ensures a fair comparison, allowing each model an equal opportunity to explore the parameter space within a given realistic timeframe.

A critical decision in our search strategy was balancing the trade-off between search breadth (the total number of hyperparameter combinations tested) and evaluation fidelity (the reliability of the performance metric for each combination). With our fixed 30-minute budget (Wall Clock Time), increasing the number of epochs per trial reduces the total number of trials Optuna can perform. While a shorter duration of 3 epochs would have allowed for a broader exploration of the search space, we opted for 5 epochs per trial.

Since we are utilizing Transfer Learning with pre-trained weights, the models converge relatively quickly. However, the first few epochs can often be noisy or dominated by the initial adaptation of the classifier head. Extending the trial to 5 epochs provided a more stable and reliable convergence trajectory, significantly reducing the risk of discarding promising hyperparameters due to transient fluctuations (early-training noise) while still maintaining a sufficient volume of trials to effectively explore the solution space. This configuration allowed Optuna to complete 3 trials for VGG-19 and 3 trials for AlexNet within the allotted time.

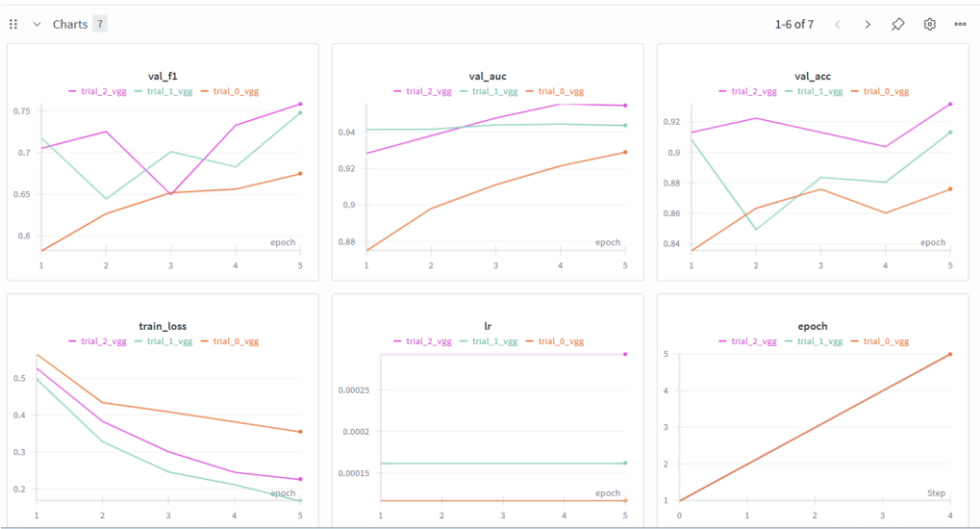
Analysis of W&B Graphs:

We maximized the Validation F1-Score as our objective metric rather than Accuracy. Given the class imbalance, a model predicting "Not Van Gogh" for every image would still achieve ~84%

accuracy but fail the task entirely. The F1-Score provided a balanced measure of Precision and Recall. Our search space explored Learning Rates from 10^{-5} to 10^{-3} (log scale), Batch Sizes of 16, 32, and 64, and compared Adam against SGD.

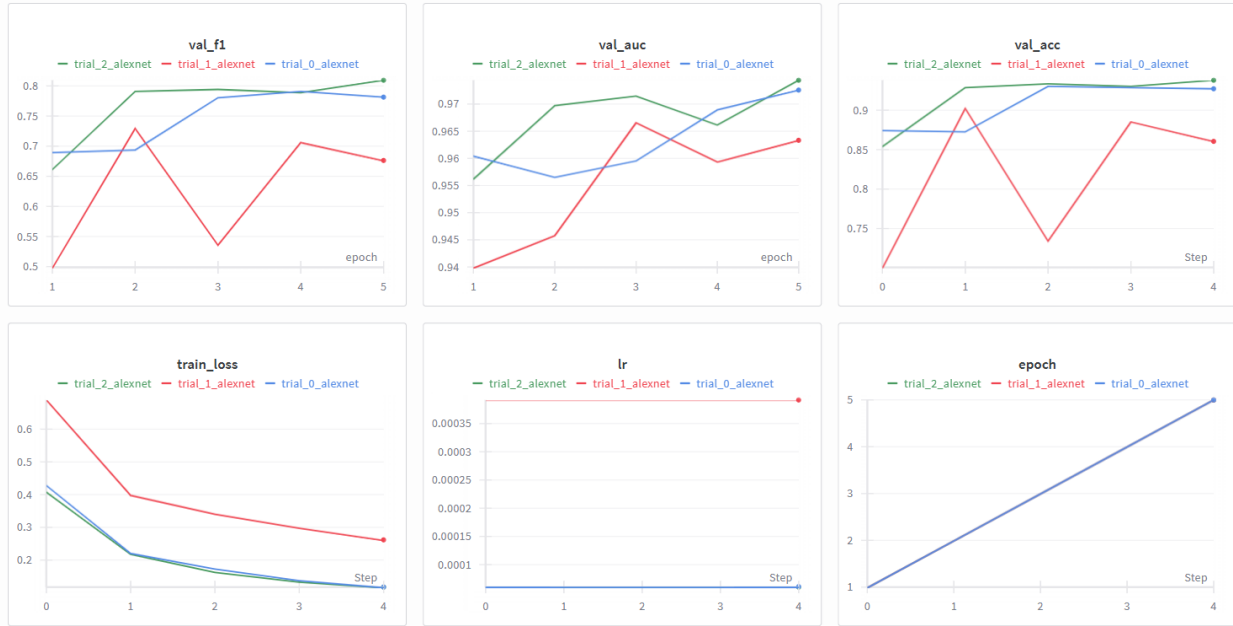
VGG:

☐ NAME	3 visualized	STATE	NOTES	US	TAI	CRE	RUNTIME	SWEET	BATCH	LEARN	OPTIM	BATCH	EPOCH	LR	TRAIN	VAL_A	VAL_A	VAL_F
☐ trial_2_vgg		Finished	Add n...	guygali		2w ago	12m 29s	-	32	0.0002938	Adam	32	5	0.0002938	0.22667	0.93178	0.95459	0.75824
☐ trial_1_vgg		Finished	Add n...	guygali		2w ago	12m 59s	-	32	0.0001620	Adam	32	5	0.0001620	0.16842	0.91318	0.94365	0.74775
☐ trial_0_vgg		Finished	Add n...	guygali		2w ago	12m 38s	-	32	0.0001165	SGD	32	5	0.0001165	0.35552	0.87597	0.929	0.6748



AlexNet:

☐ trial_2_alexnet	Finished	Add n...	guygali	2w ago	9m 46s	-	32	0.0000601	Adam	32	5	0.0000601	0.11731	0.93798	0.97439	0.80952
☐ trial_1_alexnet	Finished	Add n...	guygali	2w ago	11m 6s	-	16	0.0003918	Adam	16	5	0.0003918	0.26175	0.86047	0.96333	0.67626
☐ trial_0_alexnet	Finished	Add n...	guygali	2w ago	10m 38s	-	32	0.0000609	Adam	32	5	0.0000609	0.11759	0.92713	0.97257	0.7814



Optimizer Dominance: The Adam optimizer (represented by the lines converging towards higher F1 scores) consistently outperformed SGD. The trials using SGD often failed to converge or settled at suboptimal local minima within the limited epoch count.

Learning Rate Sensitivity: There is a clear "sweet spot" for the Learning Rate. For VGG-19, values around 10^{-4} yielded the highest stability and performance. AlexNet, being a shallower network with different gradient dynamics, favored a slightly lower learning rate in the range of 10^{-5} . Values outside these ranges led to either slow convergence (too low) or unstable loss oscillation (too high). Learning Rate was by far the most influential hyperparameter, accounting for the majority of the variance in the Validation F1-Score. This justifies our fine-grained tuning of this specific parameter in the final training phase.

Batch Size: While less critical than the learning rate, a Batch Size of 32 offered the best balance between training stability and generalization, consistently appearing in the top-performing trials.

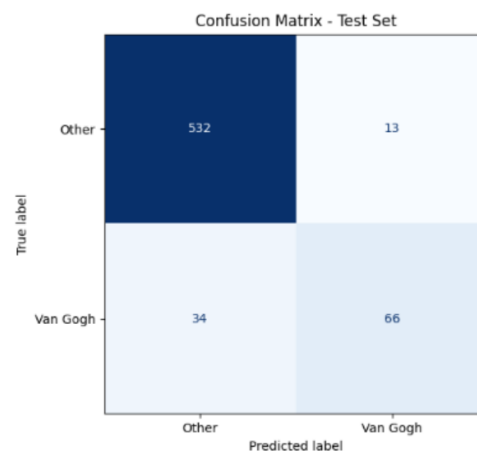
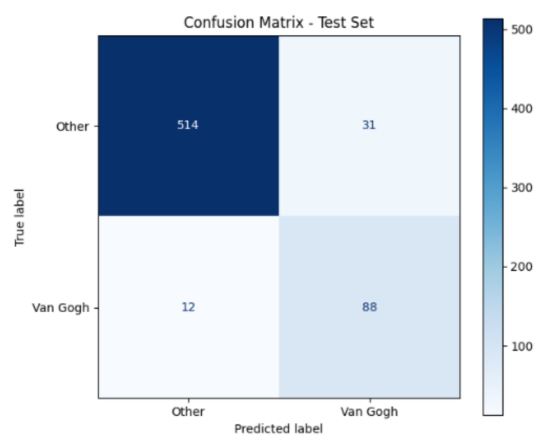
Final Configuration: Based on these insights, we selected the best trial from each 30-minute session as our final configuration for full training:

Model	Optimizer	Learning Rate	Batch Size	Class Weights
VGG-19	Adam	2.94 times 10^{-4}	32	[1.0, 5.0]
AlexNet	Adam	6.01 times 10^{-5}	32	[1.0, 5.0]

Evaluation & Model Selection

Following the hyperparameter search, we retrained both models on the full training set for 5 epochs using the optimal settings found. Upon evaluating both models on the unseen Test Set, a compelling trade-off emerged between general detection capability and classification strictness. AlexNet achieved superior overall performance metrics, with an Accuracy of 0.9333, an F1-Score of 0.8037, and an impressive ROC-AUC of 0.9679. In comparison, VGG-19 posted slightly lower general stats, with an Accuracy of 0.9271, an F1-Score of 0.7374, and an ROC-AUC of 0.9280.

However, for the specific purpose of this project-serving as a discriminator ("Judge") for Style Transfer- Precision is the most critical metric. We require a model that is "strict" and difficult to fool, rather than one that casts a wide net. VGG-19 demonstrated significantly higher precision (calculated using the Confusion matrixes) (~83.5%) compared to AlexNet (~74%). This means VGG-19 rarely misclassifies a non-Van Gogh painting as authentic, whereas AlexNet is more permissive. Consequently, VGG-19 was selected as the superior Judge. Its conservative nature forces the Style Transfer algorithm to generate high-fidelity features to achieve a passing score, whereas AlexNet's higher False Positive rate suggests it could be easily satisfied by "imposter" images that lack genuine stylistic depth. Also can be seen from the Confusion Matrixes (AlexNet on the left and VGG-19 on the right):



The confusion matrices provide a granular breakdown of this behavioral difference. VGG-19 acted as a conservative gatekeeper: it correctly rejected the vast majority of non-Van Gogh images, producing only 13 False Positives (Imposters). However, this strictness came at the cost of Recall, as it missed 34 actual Van Gogh paintings (False Negatives), correctly identifying only 66 True Positives.

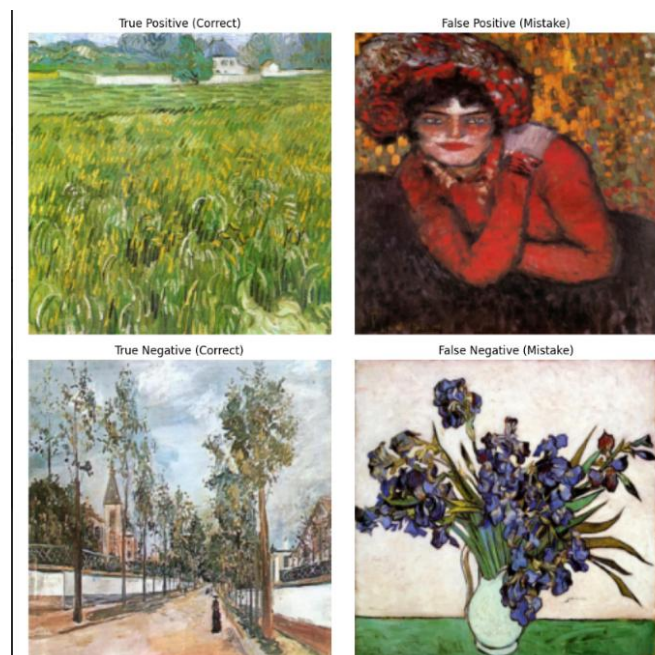
In contrast, AlexNet acted as a high-recall detector. It successfully identified 88 True Positives, missing only 12 actual Van Goghs. However, this sensitivity led to a significantly higher error rate on the negative class, allowing 31 False Positives through, nearly triple the amount of VGG-19.

Error Analysis:

False Positives ("The Imposters"): The 31 images that fooled AlexNet (but were mostly caught by VGG) are likely paintings by other Post-Impressionists with similar color palettes but different brushwork textures. AlexNet's shallower architecture likely latched onto broad color features, while VGG's depth allowed it to spot the textural inconsistency and reject them.

False Negatives ("The Missed"): The 34 Van Goghs missed by VGG (compared to only 12 missed by AlexNet) likely represent deviations from Van Gogh's "canonical" style-such as darker sketches or early works lacking his signature impasto swirls. VGG's specialized feature detectors failed to trigger on these outliers, while AlexNet's broader filters successfully captured them.

Examples of True Positives, True Negatives, False Positives and False Negatives for each model: (AlexNet on the left, VGG on the right)



Part 2: Discussion

Style Transfer Process

Neural Style Transfer (NST) is implemented as an optimization technique that synthesizes a new image by fusing the semantic structure of a content photograph with the artistic texture of a style painting. Unlike traditional deep learning, where model weights are updated to minimize loss, here we utilize a pre-trained CNN (such as VGG-19 or AlexNet) as a fixed feature extractor. The process initializes a "Generated Image" (typically as a copy of the content) and iteratively updates its pixel values using backpropagation, while the network parameters themselves remain frozen.

The core of this process drives the generated image toward a specific visual objective defined by a composite loss function. To preserve the subject matter, Content Loss minimizes the Mean Squared Error (MSE) between feature maps at deeper network layers (e.g., conv3_2), capturing high-level shapes while ignoring exact pixel constraints. Simultaneously, Style Loss captures artistic texture independent of spatial arrangement. This is achieved by calculating the Gram Matrix for multiple layers, which represents correlations between feature filters, effectively encoding brushstrokes and color palettes, and minimizing the weighted MSE between the style and generated images.

To ensure the final output is visually coherent, we incorporated Total Variation (TV) Loss as a critical regularizer. Standard style transfer can often yield high-frequency noise or "checkerboard" artifacts because pixels are optimized independently. TV Loss penalizes large intensity differences between neighboring pixels, encouraging spatial smoothness and natural-looking transitions. The optimization is executed using the Adam optimizer over a fixed number

of steps (e.g., 300), which dynamically balances these three loss components- content, style, and regularization based on the specific hyperparameters identified in our Optuna search.

```
# =====
# 4.Style Transfer
# =====
def run_style_transfer_single(content_tensor, style_tensor, c_weight, s_weight, tv_weight, lr,
                             layer_weights_dict, content_layer_name, num_steps=NUM_STEPS):
    if content_tensor.shape != style_tensor.shape:
        style_tensor = F.interpolate(style_tensor, size=content_tensor.shape[-2:], mode='bilinear')

    target = content_tensor.clone().requires_grad_(True).to(DEVICE)
    optimizer = optim.Adam([target], lr=lr)
    scheduler = optim.lr_scheduler.StepLR(optimizer, step_size=100, gamma=0.5)

    layers = {
        '0': 'conv1_1', '5': 'conv2_1', '10': 'conv3_1', '12': 'conv3_2',
        '19': 'conv4_1', '21': 'conv4_2', '28': 'conv5_1', '30': 'conv5_2'
    }

    content_features = get_features(content_tensor, vgg_extractor, layers)
    style_features = get_features(style_tensor, vgg_extractor, layers)
    style_grams = {layer: gram_matrix(style_features[layer]) for layer in style_features if layer in layer_weights_dict}

    for i in range(num_steps):
        target_features = get_features(target, vgg_extractor, layers)

        # 1. Content Loss
        c_loss = torch.mean((target_features[content_layer_name] - content_features[content_layer_name]) ** 2)
```

```
    # 2. Style Loss
    s_loss = 0
    for layer_name, weight in layer_weights_dict.items():
        if layer_name in target_features:
            target_gram = gram_matrix(target_features[layer_name])
            style_gram = style_grams[layer_name]
            b, d, h, w = target_features[layer_name].shape
            layer_s_loss = weight * torch.mean((target_gram - style_gram) ** 2)
            s_loss += layer_s_loss / (d * h * w)

    # 3. Total Variation Loss
    diff_i = torch.sum(torch.abs(target[:, :, :, 1:] - target[:, :, :, :-1]))
    diff_j = torch.sum(torch.abs(target[:, :, 1:, :] - target[:, :, :-1, :]))
    tv_loss = (diff_i + diff_j) / (target.nelement())

    # Loss
    total_loss = c_weight * c_loss + s_weight * s_loss + tv_weight * tv_loss

    optimizer.zero_grad()
    total_loss.backward()
    optimizer.step()
    scheduler.step()

    return target
```

Style Loss Normalization

Normalization of the Style Loss is fundamental to ensuring the stability and visual quality of the style transfer process. Neural networks inherently function as "pyramids," where early layers possess large spatial dimensions (H times W) with fewer filters, while deeper layers have smaller dimensions but a significantly higher number of filters. The raw values of the Gram

Matrix, calculated as the dot product of vectorized feature maps, scale quadratically with the total number of elements in a layer. Without normalization, this disparity leads to imbalanced contributions, where layers with larger dimensions produce exponentially larger loss values. Consequently, the optimization process becomes dominated by a single layer, ignoring the multi-scale style details provided by others, and often causing "gradient explosion", which results in numerical instability or "burnt" images with saturated pixel values.

To mitigate these issues, our approach implements a strict normalization protocol within the loss function. We normalize the Style Loss by dividing the sum of squared errors by the total volume of the feature map ($D \times H \times W$) squared. This operation ensures that the magnitude of the loss remains invariant to the architectural dimensions of the specific network layer. By decoupling the loss magnitude from the layer size, we guarantee that the relative influence of each layer on the final image is determined solely by the explicit hyperparameters (weights) we defined or optimized via Optuna, rather than by arbitrary structural properties of the CNN.

H = Height: The vertical spatial dimension (number of pixels) of the feature map.

W = Width: The horizontal spatial dimension (number of pixels) of the feature map.

D = Depth: The number of Channels or Filters in that specific layer

Hyperparameter Search Strategy

To determine the optimal equilibrium between preserving content structure and imposing artistic stylization, we conducted a systematic hyperparameter search for the style transfer process. Unlike the classification phase in Part 1, our objective function here was custom-designed to maximize the "Van Gogh Probability" score assigned by our pre-trained classifiers acting as judges. Utilizing Optuna, we executed 30 trials for each generator architecture (VGG-19 and AlexNet), exploring a comprehensive search space that included Learning Rate (0.01 - 0.1), Total Variation (TV) Weight (10^{-6} - 10^{-3}), and individual weights for specific convolutional layers. Crucially, the Content Weight (α) was fixed at 1.0 to serve as a constant baseline, while the Style Weight (β) was searched on a logarithmic scale from 10^4 to 10^9 to identify the necessary magnitude for effective stylization.

The optimization process revealed that a significantly high Style Weight was indispensable for satisfying the classifiers, likely due to the numerical disparity between the unnormalized Gram Matrix differences and the Content Loss.

For VGG-19, the optimal configuration converged at a massive style weight of β in approx 6.51×10^8 with a TV weight of 1.13×10^{-5} .

Similarly, AlexNet required a substantial style weight of β approx 9.66×10^7 with a TV weight of 9.12×10^{-5} . These values indicate that the "judges" require a dominant texture signal to recognize the artwork as authentic, necessitating a style loss magnitude that far exceeds the content loss in the optimization landscape.

Parameter	Symbol	VGG-19 Value	AlexNet Value
Content Weight	alpha	1.0\$	1.0
Style Weight	beta	6.52 times 10 ⁸	9.66 times 10 ⁷
TV Weight	TV	\$1.13 times 10 ^{-5}	\$9.12 times 10 ^{-5}

Classifier Evaluation

In Part 2C, we generated 20 images using each model and cross-evaluated them using both classifiers acting as "Judges." This created an adversarial dynamic, allowing us to test if an image generated by VGG could "fool" the AlexNet judge, and vice versa. The results revealed distinct preferences: the VGG-19 generator overwhelmingly favored the complex textures of Dr. Paul Gachet, while the AlexNet generator showed a strong bias toward the high-contrast swirls of The Starry Night.

Transfer learning scores using VGG:

Image	Best_Style_Used	Generator	Judge_VGG_Score	Judge_AlexNet_Score
001.jpg	vincent-van-gogh_dr-paul-gachet-1890(1).jpg	VGG19	0.9882	0.8749
annie-spratt-fbAnljhrOL4-unsplash.jpg	vincent-van-gogh_sower-after-millet-1889(1).jpg	VGG19	0.9886	0.065
bp-miller-rGDv4llw-mk-unsplash.jpg	vincent-van-gogh_dr-paul-gachet-1890(1).jpg	VGG19	0.9998	0.1664
cristina-gottardi-CSpjU6hYo_0-unsplash.jpg	vincent-van-gogh_dr-paul-gachet-1890(1).jpg	VGG19	0.9892	0.9922
daniele-franchi-	vincent-van-gogh_dr-paul-	VGG19	0.9917	0.971

cuPoa2L52-l-unsplash.jpg	gachet-1890(1).jpg			
diego-ph-flq0tET6llw-unsplash.jpg	vincent-van-gogh_the-starry-night-1889(1).jpg	VGG19	0.983	0.7797
ed-wingate-Q-7sbt4npbg-unsplash.jpg	vincent-van-gogh_dr-paul-gachet-1890(1).jpg	VGG19	0.9855	0.4345
eric-prouzet-5lUMTeo7-bE-unsplash.jpg	vincent-van-gogh_dr-paul-gachet-1890(1).jpg	VGG19	0.9949	0.0134
jakob-owens-esbfaHABh7Y-unsplash.jpg	vincent-van-gogh_dr-paul-gachet-1890(1).jpg	VGG19	0.9987	0.7555
jonas-svidras-bgoE05DFF9U-unsplash.jpg	vincent-van-gogh_dr-paul-gachet-1890(1).jpg	VGG19	0.9984	0.5241
joshua-kettle-sKOBxEb6E-E-unsplash.jpg	vincent-van-gogh_dr-paul-gachet-1890(1).jpg	VGG19	0.9978	0.9913
keegan-houser-Q_t4SCN8c4-unsplash.jpg	vincent-van-gogh_dr-paul-gachet-1890(1).jpg	VGG19	0.9765	0.996
kelly-jean-cKjbl0Y00X0-unsplash.jpg	vincent-van-gogh_sower-after-millet-1889(1).jpg	VGG19	0.9887	0.0868
lerone-pieters-j57b5V7_RJw-unsplash.jpg	vincent-van-gogh_dr-paul-gachet-1890(1).jpg	VGG19	0.9776	0.9807
markus-spiske-ANrVttbNss-unsplash.jpg	vincent-van-gogh_dr-paul-gachet-1890(1).jpg	VGG19	0.9946	0.521
mehedi-hasan-hurr07ccyEk-unsplash.jpg	vincent-van-gogh_dr-paul-gachet-1890(1).jpg	VGG19	0.9999	0.7685
mike-meyers-haAxbjiHds-unsplash.jpg	vincent-van-gogh_dr-paul-gachet-1890(1).jpg	VGG19	0.9945	0.0081
muhammad-kamran-khan-WMp_KnaTYSs-unsplash.jpg	vincent-van-gogh_dr-paul-gachet-1890(1).jpg	VGG19	0.9838	0.9552
nikolai-lehmann-SjNe0bwcuyw-unsplash.jpg	vincent-van-gogh_dr-paul-gachet-1890(1).jpg	VGG19	0.9977	0.8672
peiwen-yu-Etpd8Le6b8E-unsplash.jpg	vincent-van-gogh_the-starry-night-1889(1).jpg	VGG19	0.9922	0.2542

Transfer learning using ALEXNET:

Image	Best_Style_Used	Generator	Judge_VGG_Score	Judge_AlexNet_Score
001.jpg	vincent-van-gogh_the-starry-night-1889(1).jpg	ALEXNET	0.7958	0.9997
annie-spratt-fbAnljhrOL4-unsplash.jpg	vincent-van-gogh_a-pair-of-shoes-1888(1).jpg	ALEXNET	0.0564	0.5816
bp-miller-rGDv4llw-mk-unsplash.jpg	vincent-van-gogh_japanese-vase-with-roses-and-anemones-1890(1).jpg	ALEXNET	0.6379	0.3875
cristina-gottardi-CSpjU6hYo_0-unsplash.jpg	vincent-van-gogh_the-starry-night-1889(1).jpg	ALEXNET	0.7434	0.9968
daniele-franchi-cuPoa2L52-l-unsplash.jpg	vincent-van-gogh_the-starry-night-1889(1).jpg	ALEXNET	0.4727	0.8962
diego-ph-flq0tET6llw-unsplash.jpg	vincent-van-gogh_the-starry-night-1889(1).jpg	ALEXNET	0.9987	0.9529
ed-wingate-Q-7sbt4npbg-unsplash.jpg	vincent-van-gogh_the-starry-night-1889(1).jpg	ALEXNET	0.8924	0.9752
eric-prouzet-5lUMTeo7-bE-unsplash.jpg	vincent-van-gogh_japanese-vase-with-roses-and-anemones-1890(1).jpg	ALEXNET	0.5247	0.0095
jakob-owens-esbfaHABh7Y-unsplash.jpg	vincent-van-gogh_dr-paul-gachet-1890(1).jpg	ALEXNET	0.9895	0.9853
jonas-svidras-bgoE05DFF9U-unsplash.jpg	vincent-van-gogh_the-starry-night-1889(1).jpg	ALEXNET	0.4623	0.9525
joshua-kettle-sKOBxEb6E-unsplash.jpg	vincent-van-gogh_the-starry-night-1889(1).jpg	ALEXNET	0.9156	1
keegan-houser-Q_t4SCN8c4-unsplash.jpg	vincent-van-gogh_the-starry-night-1889(1).jpg	ALEXNET	0.2253	1
kelly-jean-cKjbl0Y00X0-unsplash.jpg	vincent-van-gogh_the-starry-night-1889(1).jpg	ALEXNET	0.9173	0.889
lerone-pieters-j57b5V7_RJw-unsplash.jpg	vincent-van-gogh_a-pair-of-shoes-1888(1).jpg	ALEXNET	0.1376	0.9962
markus-spiske-ANrVttbNss-unsplash.jpg	vincent-van-gogh_the-starry-night-1889(1).jpg	ALEXNET	0.8891	0.7897
mehedi-hasan-	vincent-van-gogh_the-	ALEXNET	0.509	0.9973

hurr07ccyEk-unsplash.jpg	starry-night-1889(1).jpg	ET		
mike-meyers-haAxbjiHds-unsplash.jpg	vincent-van-gogh_a-pair-of-shoes-1888(1).jpg	ALEXN ET	0.293	0.8938
muhammad-kamran-khan-WMp_KnaTYSs-unsplash.jpg	vincent-van-gogh_the-starry-night-1889(1).jpg	ALEXN ET	0.9709	0.9998
nikolai-lehmann-SjNe0bwcuyw-unsplash.jpg	vincent-van-gogh_the-starry-night-1889(1).jpg	ALEXN ET	0.4985	0.8437
peiwen-yu-Etpd8Le6b8E-unsplash.jpg	vincent-van-gogh_the-starry-night-1889(1).jpg	ALEXN ET	0.7927	0.7661

Consensus (Both classified as "Van Gogh"):



In several instances, the style transfer was robust enough to satisfy both architectures. For example, the VGG-generated image of “joshua-kettle-sKOBxEb6E-E-unsplash.jpg” (applying Dr. Paul Gachet) achieved a 0.9978 score from the VGG judge and a 0.9913 from the AlexNet judge. Similarly, the AlexNet-generated “ muhammad-kamran-khan-WMp_KnaTs-unsplash.jpg” (applying The Starry Night) satisfied both, with scores of 0.97 (VGG) and 0.99 (AlexNet). These cases represent the "sweet spot" where the generated features-likely strong brushstroke textures combined with distinct color contrasts-aligned with the feature maps of both shallow and deep networks.

Disagreement (VGG yes, ALEXNET no):



This scenario highlights the superior depth of VGG. A striking example is the VGG-generated eric-prouzet-5lUMTeo7-bE-unsplash.jpg (using Dr. Paul Gachet). The VGG judge was nearly certain it was a Van Gogh (0.9949), while the AlexNet judge rejected it entirely (0.0134). This suggests that VGG optimized for complex, high-frequency textural details characteristic of the Gachet portrait. The deeper VGG network could resolve these fine details, but the shallower AlexNet likely perceived them as mere noise rather than artistic style, leading to a rejection.

Disagreement (VGG No, AlexNet Yes):



Conversely, when AlexNet acted as the generator, it often produced images that satisfied itself but failed the VGG test. For instance, “keegan-houser--Q_t4SCN8c4-unsplash.jpg” (using The Starry Night) received a perfect 1.0 from the AlexNet judge but a low 0.2253 from VGG. This indicates that AlexNet optimization often converges on "superficial" style features-such as broad swirls or high-contrast edges, that trigger its own filters. However, these features lacked the intricate, multi-scale coherence required to pass the stricter VGG discriminator.

Style Selection Bias:

An analysis of the chosen styles reveals a clear pattern. The VGG Generator almost exclusively selected Dr. Paul Gachet (characterized by chaotic, fine brushstrokes), aligning with VGG's ability to capture high-frequency texture. In contrast, the AlexNet Generator preferred The Starry Night (characterized by large, distinct shapes and swirls), aligning with AlexNet's reliance on broader, shape-based features (large receptive fields). This confirms that the architecture of the neural network dictates not just how it transfers style, but which artistic features it considers dominant.

Layer Selection

The choice of layers for feature extraction is fundamental to the quality of the result.

Content Layer: We selected deeper layers - conv3_2 for VGG-19 and conv3 for AlexNet. Early layers (e.g., conv1) capture exact pixel values and edges. Constraining the content at this level would force the generated image to be a near-replica of the photograph, leaving no room for artistic distortion. Deeper layers capture the semantic structure and arrangement of objects while being invariant to exact pixel details, allowing the algorithm to "repaint" the content in a new style without losing the subject.

Style Layers: We aggregated the loss from a hierarchy of layers: conv1 through conv5. Artistic style manifests at multiple scales. Lower layers capture localized brushstrokes, textures, and color palettes. Higher layers capture larger motifs, shapes, and spatial composition. Using a weighted sum of all these layers ensures the generated image replicates the global "feel" of the artwork, not just its colors.

Overall Reflection

The results from Part 2 align strongly with our findings in Part 1. In the classification phase, VGG-19 emerged as the superior and more precise model (higher Precision, F1-Score of 0.71), while AlexNet was more prone to false positives (lower Precision).

This distinction carried over into generation. The images generated by the VGG-based style transfer were generally more visually coherent and complex, reflecting the model's deeper understanding of texture. Furthermore, VGG acted as a stricter "Judge," requiring higher quality features to output a high probability score. In contrast, AlexNet's generated images were often noisier or simpler, and as a judge, it was inconsistent—sometimes rejecting high-quality VGG images or accepting noisy ones. This confirms that a model's capacity to classify complex patterns (depth) directly correlates with its ability to guide the generation of those patterns.

Discussion : Hardware Acceleration Analysis: GPU vs. CPU

To quantify the impact of hardware acceleration on our training pipeline, we conducted a comparative benchmark between CPU and GPU execution. The experiment consisted of training a single full epoch (5,160 images) with a batch size of 32, corresponding to approximately 162 backpropagation iterations.

```

✅ Device detection: cuda
cuda available: True
GPU Device: NVIDIA GeForce RTX 3080
Python version: 3.12.10
Machine name: wol17-424-pc
Working directory: C:\Users\yairshapira\PyCharmMiscProject
Initial GPU memory usage:
0.0 MB allocated
0.0 MB reserved
📁 Total images found: 6450
📁 Dataset Size for Benchmark: 5160

🚀 Starting benchmark for 1 Epoch on cpu (Option 1 Benchmark)...
Training on cpu: 100%|██████████| 162/162 [02:39<00:00, 1.02it/s]
🕒 Finished 1 Epoch on cpu. Time: 159.33 seconds.

🚀 Starting benchmark for 1 Epoch on cuda (Option 1 Benchmark)...
Training on cuda: 100%|██████████| 162/162 [02:15<00:00, 1.19it/s]
🕒 Finished 1 Epoch on cuda. Time: 135.68 seconds.
```

Per

Con Final Benchmark Results (Option 1): The GPU yielded a modest
spe CPU Epoch Time: 159.33s (2.66m) faster, this result suggests
tha GPU Epoch Time: 135.68s (2.26m) work in this configuration
like Speedup Factor: 1.17x faster on GPU mathematical calculation.

When the model is relatively light or the batch size is small, the time spent fetching data from the disk and transferring it to the GPU memory (VRAM) can dominate the total execution time, effectively masking the GPU's raw parallel processing capability.

GPU activity during training- Utilization Analysis

```
PS C:\Users\yairshapira\PyCharmMiscProject> nvidia-smi
Sun Jan 11 13:12:42 2026

+-----+
| NVIDIA-SMI 580.97                Driver Version: 580.97          CUDA Version: 13.0     |
+-----+-----+
| GPU   Name                Driver-Model  | Bus-Id        Disp.A | Volatile Uncorr. ECC | | |
| Fan   Temp   Perf          Pwr:Usage/Cap |      Memory-Usage | GPU-Util  Compute M. |
|               |                    |              |      |               |
+-----+-----+
|  0  NVIDIA GeForce RTX 3080    WDDM        | 00000000:01:00.0 Off |              N/A     | | |
| 0%    56C    P2              91W / 320W | 3208MiB / 10240MiB |    21%    Default   |
|               |                    |              |      |               |
+-----+-----+

+-----+
| Processes:                        |
| GPU   GI    CI          PID    Type   Process name                  GPU Memory |
|               |               |                    |          | Process name                  | Usage     |
+-----+-----+
|  0   N/A   N/A         42280    C     ...am Files\Python312\python.exe  N/A       |
+-----+-----+
```

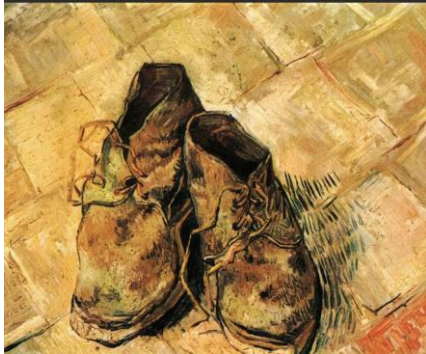
A closer inspection of the GPU telemetry during training provides the technical explanation for this limited speedup. The average GPU utilization hovered at a mere 21%, indicating that the computation cores were idle for nearly 80% of the execution time, likely waiting for data fetching and pre-processing (I/O bottleneck). Furthermore, VRAM consumption was recorded at 3,208 MiB - approximately 31% of the 10,240 MiB available capacity. This substantial underutilization confirms that the current workload configuration was insufficient to saturate the hardware's parallel processing capabilities, leaving the device capable of handling significantly heavier compute demands.

Scalability Implications

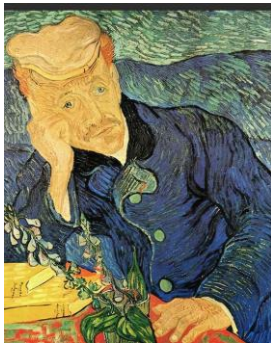
This utilization data highlights that the performance gap is highly sensitive to the arithmetic intensity of the task. Since the GPU was largely idle, its theoretical advantage was masked by the data transfer overhead. The GPU's true potential becomes significantly more pronounced in scenarios requiring denser computations. Had we utilized a larger batch size, executed a more prolonged training schedule, or employed a deeper architecture (requiring significantly more FLOPs per byte of data), the ratio of computation to data transfer would have increased. Under such conditions, we would expect the GPU utilization to rise significantly above 21%, likely resulting in a much more dramatic divergence in performance compared to the CPU.

Appendix A:

We used these 5 Van Gogh images for the image generation process:
vincent-van-gogh_a-pair-of-shoes-1888



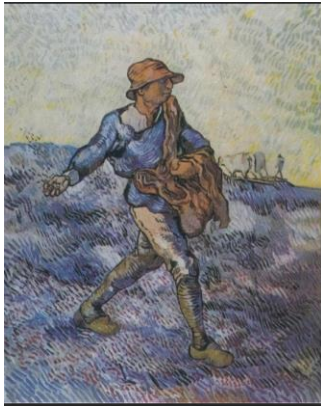
vincent-van-gogh_dr-paul-gachet-1890



vincent-van-gogh_japanese-vase-with-roses-and-anemones-1890



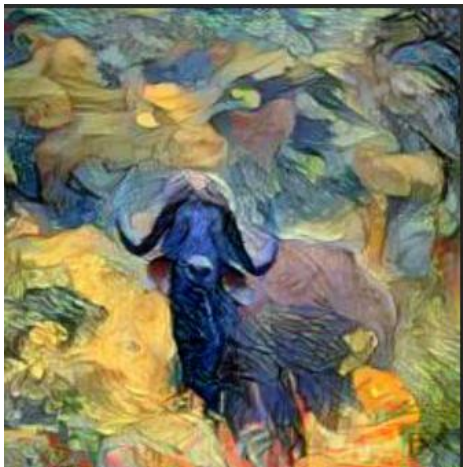
vincent-van-gogh_sower-after-millet-1889

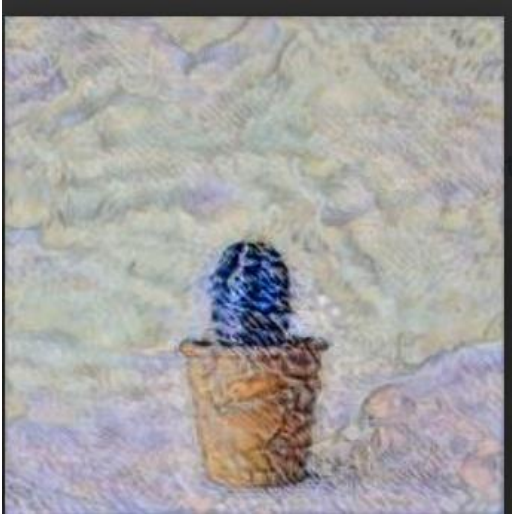


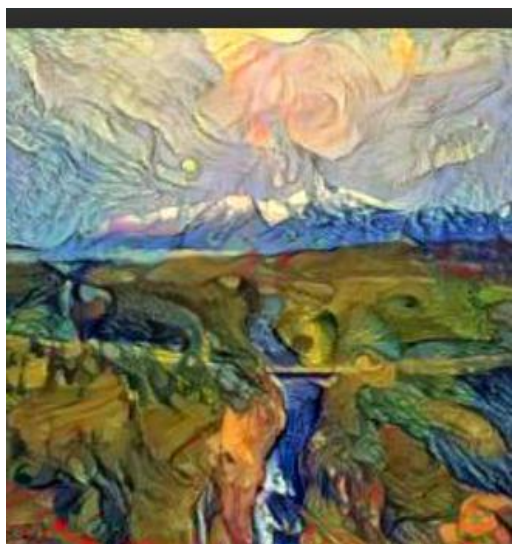
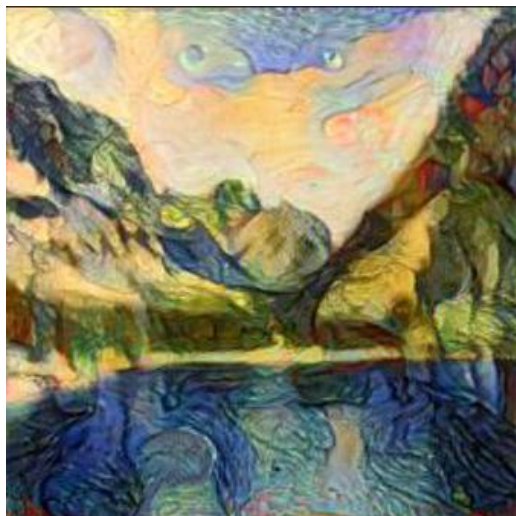
vincent-van-gogh_the-starry-night-1889

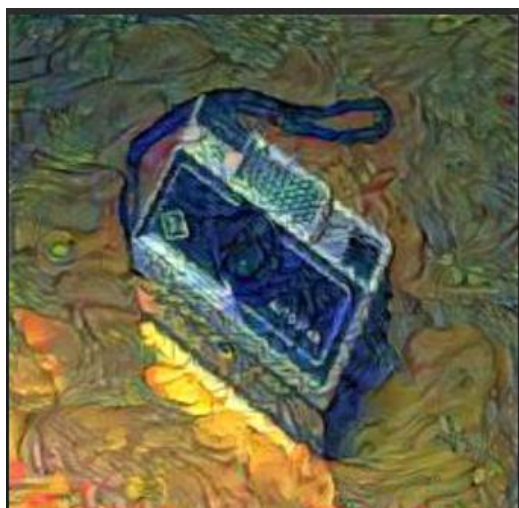


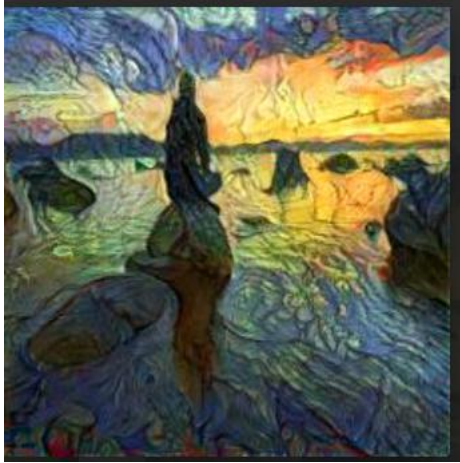
Appendix B- VGG generated images:

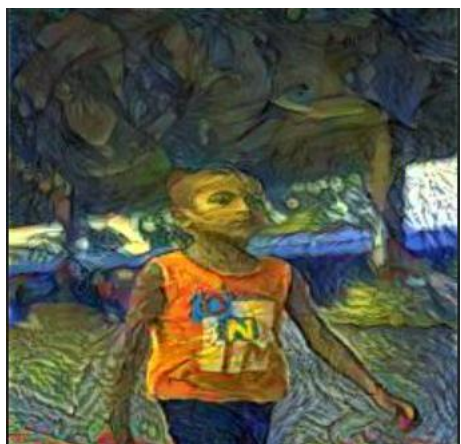


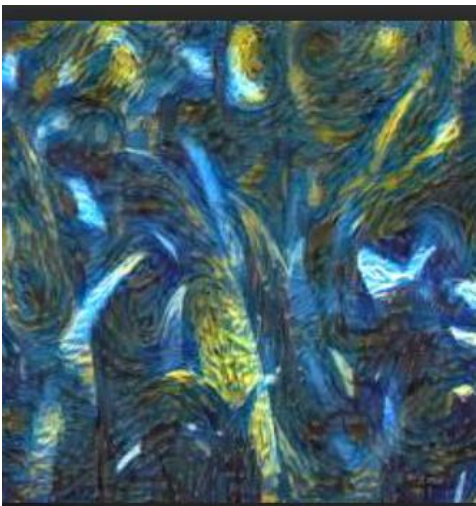














Appendix C- ALEXNET generated images:



